paiement pawapay :

Tout ce qui concern le depot, nous recevons de l’ argent de l’ user qui veut s’ abonner

Introduction

Par où commencer ?

Lors de la création d'un compte pawaPay, vous aurez d'abord accès à notre environnement de test. Cet environnement est totalement isolé de notre environnement de production.

Vous pouvez tester en toute sécurité votre intégration avec pawaPay sans utiliser de portefeuilles de monnaie mobile réels ni d'argent réel, immédiatement après la création de votre compte pawaPay.

L'accès à votre compte de production, sur lequel les paiements réels peuvent être effectués, vous sera accordé une fois l'intégration sur votre compte sandbox terminée.

Maintenant que vous avez un compte dans notre environnement de test, voyons comment commencer l'intégration.

L'URL de base de l'API marchand pawaPay est différente entre nos environnements de test et de production.

ac à sable tps://api.sandbox.pawapay.io/

·oduction tps://api.pawapay.io/

L'opération spécifique peut être appelée en ajoutant le point de terminaison à l'URL de base.

Exemple : https://api.sandbox.pawapay.io/v2/payouts

Les URL de base diffèrent entre les comptes de test et de production. Veuillez les enregistrer dans la configuration spécifique à l'environnement de votre application.

Depuis le tableau de bord pawaPay, vous pouvez :

- Configurer les URL de rappel
- Générer un jeton API
- Voir tous les paiements acceptés
- Consultez les états financiers
- Inviter des utilisateurs
- Niveaux d'accès au contrôle
- Et bien plus encore

Comme notre environnement de test est complètement isolé de notre environnement de production, les URL permettant d'accéder au tableau de bord pawaPay sont différentes.

ac à sable tps://dashboard.sandbox.pawapay.io/

·oduction tps://dashboard.pawapay.io/

L'API marchand pawaPay utilise un jeton porteur pour l'authentification. Un en-tête d'autorisation est requis pour tous les appels à l'API marchand pawaPay.

Le jeton peut être généré depuis le tableau de bord pawaPay. Vous trouverez les instructions à ce sujet dans la documentation du tableau de bord pawaPay .

Voici un exemple de requête de paiement avec curl :

```
curl -i -X POST \
https://api.sandbox.pawapay.io/payouts \
-H 'Authorization: Bearer <YOUR_API_TOKEN>' \
-H 'Content-Type: application/json' \
-d '{
"payoutId": "33f30946-881d-40bc-8ca2-94aa4cd467ac",
"amount": "15",
"currency": "ZMW",
"recipient": {
    "type": "MMO",
    "accountDetails": {
        "phoneNumber": "260763456789",
        "provider": "MTN_MOMO_ZMB"
        }
    }
}'
```

Les jetons d'API diffèrent entre les comptes sandbox et les comptes de production.

Lors du passage de l'environnement de test à l'environnement de production, un nouveau jeton d'API doit être généré à partir du tableau de bord pawaPay de production.

Veuillez stocker le jeton API dans l'espace de stockage sécurisé spécifique à l'environnement de votre application.

Si vous souhaitez ajouter une deuxième couche de sécurité, vous pouvez implémenter des signatures pour garantir la sécurité même en cas de fuite de votre jeton API.

L'API pawaPay étant asynchrone, nous vous recommandons d'implémenter un gestionnaire de rappel dans votre application afin de connaître le statut final de vos paiements dès leur traitement.

Vous trouverez plus d'informations à ce sujet dans la documentation du tableau de bord pawaPay .

Une fois configuré, nous vous enverrons une notification avec le statut final du paiement à l'URL de notification que vous avez définie. En savoir plus sur les notifications .

Vous êtes maintenant prêt à intégrer l'API marchand pawaPay.

Jouez à Postman

Testez et familiarisez-vous avec l'API grâce à notre collection Postman.

Dépôts

Découvrez comment vous intégrer à l'API pour être rémunéré.

Paiements

Découvrez comment intégrer le service pour envoyer de l'argent.

Remboursements

Découvrez comment gérer les remboursements.

page de paiement

Découvrez comment être payé grâce à une expérience de paiement innovante.

Vous pouvez tester l'API et le tableau de bord pawaPay Merchant avec votre compte sandbox dès votre inscription. Aucune procédure d'intégration n'est requise, car aucun argent réel n'est impliqué.

Des numéros de téléphone spéciaux sont disponibles pour simuler des cas de réussite et différents cas d'échec.

Il n'est pas possible de visualiser les flux d'autorisation (c'est-à-dire la saisie du code PIN) dans l'environnement sandbox.

Passage en direct

Lorsque vous serez prêt à passer de l'environnement de test à la production, assurez-vous d'utiliser l' URL de base et le jeton d'authentification corrects . Ce sont les seuls paramètres qui diffèrent entre l'environnement de test et la production.

Nous vous recommandons de stocker ces paramètres dans la configuration spécifique à votre environnement.

Si vous utilisez la liste blanche d'adresses IP, assurez-vous que nos adresses IP y figurent afin que nous puissions vous rappeler.

Nous vous recommandons d'activer une fonctionnalité spécifique pour votre intégration avec pawaPay afin de permettre les tests de mise en production avant le lancement sur le marché. Cela vous permettra de tester le flux de bout en bout en production et de déceler tout problème spécifique à l'environnement avant qu'il n'affecte vos clients.

Pour tester les paiements en direct, vous aurez besoin des éléments suivants pour chaque MMO que vous utilisez :

- Un numéro de téléphone associé au MMO en question.
- Un portefeuille de paiement mobile actif sur ce numéro de téléphone.
- Un solde disponible sur le portefeuille de monnaie mobile pour effectuer les tests.
- Préparez-vous à saisir le code PIN sur votre téléphone environ 1 à 20 secondes après avoir initié le paiement via notre API.

Guides

Explorez notre API avec Postman.

L'API pawaPay vous permet de payer et d'être payé par mobile money en Afrique subsaharienne.

1

Intégrez notre API à votre espace de travail dans Postman.

2

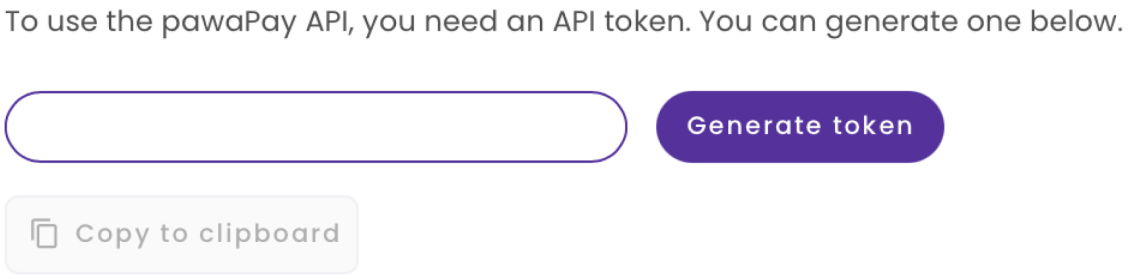
Créez un compte avec pawaPay

Si ce n'est pas déjà fait, créez un compte dans notre environnement de test. Cela ne prend que quelques minutes et vous pourrez commencer à tester notre API immédiatement.

3

Générer un jeton API

Accédez maintenant à Configuration système > Jetons d'API et générez un jeton d'API.



Veuillez le sauvegarder immédiatement. Pour des raisons de sécurité, nous ne stockons pas les jetons dans pawaPay.

4

Configurez un environnement dans Postman

1. Dans Postman, sélectionnez « Environnements » dans la barre latérale, puis cliquez sur le petit bouton « + ».
2. Nommez l'environnement comme vous le souhaitez. Par exemple : « bac à sable pawaPay ».
3. Ajoutez les variables suivantes à votre environnement :

ton API ollez le jeton que vous venez de générer comme valeur

RL de base tps://api.sandbox.pawapay.io

4. N'oubliez pas d'enregistrer vos modifications.
5. Sélectionnez l'environnement nouvellement créé dans le menu déroulant « Environnement » situé dans le coin supérieur droit.

5

Vous pouvez désormais envoyer des requêtes à l'API pawaPay dans l'environnement de test. Essayez par exemple « Initier un dépôt ».

Consultez notre documentation API pour en savoir plus sur :

Dépôts

Découvrez comment être payé.

Paiements

Découvrez comment payer quelqu'un.

Remboursements

Découvrez comment rembourser le paiement de quelqu'un.

Configuration active

Découvrez ce qui est disponible sur votre compte et tous les paramètres nécessaires pour payer et être payé.

…et bien plus encore.

L'API pawaPay est asynchrone. Le moyen le plus rapide de connaître le statut final d'un paiement est d'utiliser les rappels . Configurons-les également afin que vous puissiez visualiser le flux complet dans Postman.

1

Configurez un serveur factice dans Postman.

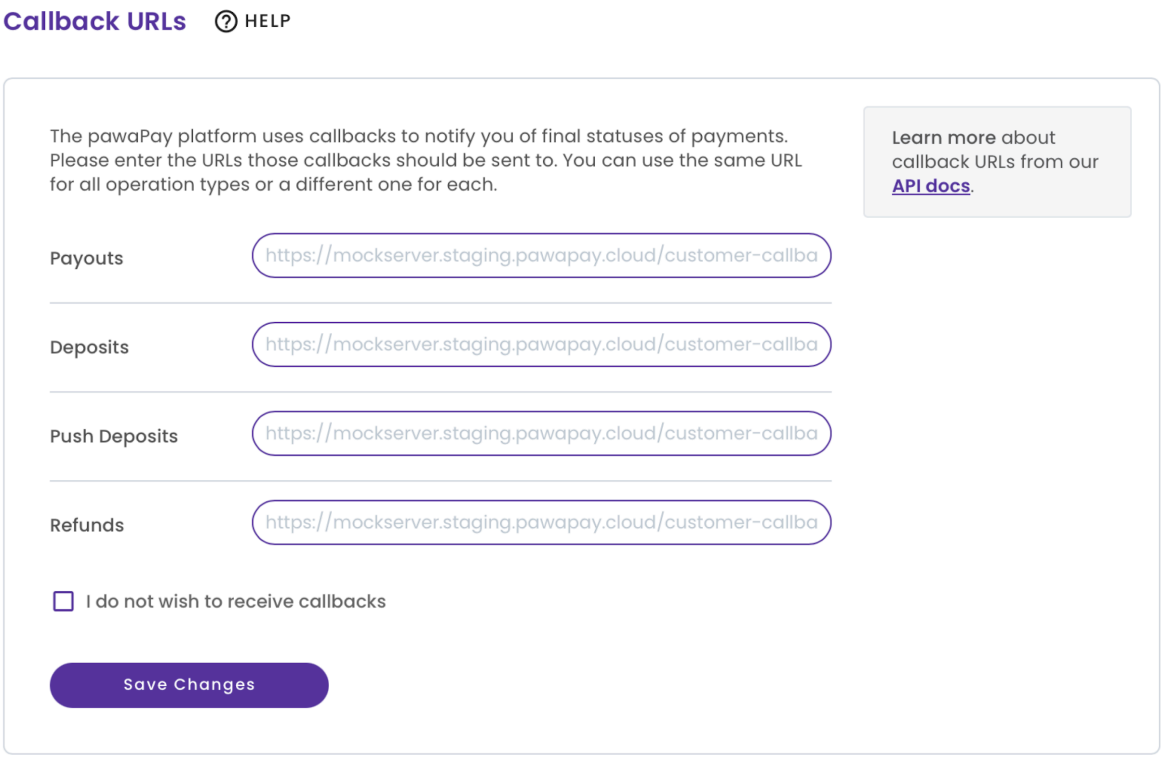
1. Choisissez « Serveurs fictifs » dans la barre latérale et sélectionnez le petit bouton « + ».
2. Choisissez « Sélectionner une collection existante ».
3. Sélectionnez « pawaPay Merchant API » comme collection à simuler.
4. Nommez le serveur comme vous le souhaitez. Par exemple : « bac à sable pawaPay ».
5. Sélectionnez « Créer un serveur factice ».
6. Copiez l'URL du serveur factice.

2

Configurer les rappels

Connectez-vous au tableau de bord sandbox et accédez à Configuration système > URL de rappel .

Sélectionnez « Modifier » et collez l'URL de rappel que vous avez copiée depuis Postman dans les champs de saisie.



Une fois tous vos flux configurés, sélectionnez « Enregistrer les modifications ».

3

C'est terminé.

Vous pouvez désormais commencer à envoyer des requêtes à l'API pawaPay dans l'environnement de test.

Vous verrez les rappels dans l'onglet Serveurs simulés en cliquant sur le serveur simulé que vous venez de créer.

N'oubliez pas de cliquer sur « Actualiser les journaux » après avoir effectué un paiement.

Consultez nos guides pour les différents flux de paiement. Vous pouvez les tester dans Postman au fur et à mesure.

Dépôts

Guide étape par étape pour se faire payer par ses clients.

Paiements

Voici les étapes à suivre pour payer vos clients.

Remboursements

Il arrive parfois qu'un remboursement soit nécessaire. Suivez ce guide pour rembourser vos clients.

page de paiement

Découvrez comment être payé grâce à une expérience de paiement innovante.

Guides
Explorez notre API avec Postman.
L'API pawaPay vous permet de payer et d'être payé par mobile money en Afrique subsaharienne.

Amusons-nous avec l'API dans Postman.
1
Intégrez notre API à votre espace de travail dans Postman.

Courir dans le facteur
2
Créez un compte avec pawaPay

Si ce n'est pas déjà fait, créez un compte dans notre environnement de test. Cela ne prend que quelques minutes et vous pourrez commencer à tester notre API immédiatement.
3
Générer un jeton API

Accédez maintenant à Configuration système > Jetons d'API et générez un jeton d'API.

Veuillez le sauvegarder immédiatement. Pour des raisons de sécurité, nous ne stockons pas les jetons dans pawaPay.
4
Configurez un environnement dans Postman

Dans Postman, sélectionnez « Environnements » dans la barre latérale, puis cliquez sur le petit bouton « + ».
Nommez l'environnement comme vous le souhaitez. Par exemple : « bac à sable pawaPay ».
Ajoutez les variables suivantes à votre environnement :

Variable	Valeur
jeton API	Collez le jeton que vous venez de générer comme valeur
URL de base	https://api.sandbox.pawapay.io

N'oubliez pas d'enregistrer vos modifications.
Sélectionnez l'environnement nouvellement créé dans le menu déroulant « Environnement » situé dans le coin supérieur droit.
5
C'est terminé !
Vous pouvez désormais envoyer des requêtes à l'API pawaPay dans l'environnement de test. Essayez par exemple « Initier un dépôt ».
Consultez notre documentation API pour en savoir plus sur :
Dépôts
Découvrez comment être payé.
Paiements
Découvrez comment payer quelqu'un.
Remboursements
Découvrez comment rembourser le paiement de quelqu'un.
Configuration active
Découvrez ce qui est disponible sur votre compte et tous les paramètres nécessaires pour payer et être payé.
…et bien plus encore.

Vous souhaitez également tester les rappels avec Postman ?
L'API pawaPay est asynchrone. Le moyen le plus rapide de connaître le statut final d'un paiement est d'utiliser les rappels . Configurons-les également afin que vous puissiez visualiser le flux complet dans Postman.
1
Configurez un serveur factice dans Postman.

Choisissez « Serveurs fictifs » dans la barre latérale et sélectionnez le petit bouton « + ».
Choisissez « Sélectionner une collection existante ».
Sélectionnez « pawaPay Merchant API » comme collection à simuler.
Nommez le serveur comme vous le souhaitez. Par exemple : « bac à sable pawaPay ».
Sélectionnez « Créer un serveur factice ».
Copiez l'URL du serveur factice.
2
Configurer les rappels

Connectez-vous au tableau de bord sandbox et accédez à Configuration système > URL de rappel .
Sélectionnez « Modifier » et collez l'URL de rappel que vous avez copiée depuis Postman dans les champs de saisie.

Une fois tous vos flux configurés, sélectionnez « Enregistrer les modifications ».
3
C'est terminé.

Vous pouvez désormais commencer à envoyer des requêtes à l'API pawaPay dans l'environnement de test.
Vous verrez les rappels dans l'onglet Serveurs simulés en cliquant sur le serveur simulé que vous venez de créer.
N'oubliez pas de cliquer sur « Actualiser les journaux » après avoir effectué un paiement.

Que faire ensuite ?
Consultez nos guides pour les différents flux de paiement. Vous pouvez les tester dans Postman au fur et à mesure.
Dépôts
Guide étape par étape pour se faire payer par ses clients.
Paiements
Voici les étapes à suivre pour payer vos clients.
Remboursements
Il arrive parfois qu'un remboursement soit nécessaire. Suivez ce guide pour rembourser vos clients.
page de paiement
Découvrez comment être payé grâce à une expérience de paiement innovante.

Dépôts

Les dépôts vous permettent de demander un paiement à un client. Les fonds seront transférés du portefeuille mobile du client vers votre compte pawaPay.

Dans ce guide, nous allons passer en revue étape par étape la manière d'aborder la mise en place d'un flux de dépôts de haute qualité.

Si ce n'est déjà fait, consultez les informations suivantes pour bien préparer votre utilisation de ce guide.

Commençons par tout paramétrer manuellement. Tout au long de ce guide, nous dynamiserons progressivement le flux et améliorerons l'expérience client.

1

Initier le dépôt

Envoyons cette charge utile au point de terminaison d'initiation de dépôt .

POST https://api.sandbox.pawapay.io/v2/deposits

```
{
    "depositId": "afb57b93-7849-49aa-babb-4c3ccbfe3d79",
    "amount": "100",
    "currency": "RWF",
    "payer": {
        "type": "MMO",
        "accountDetails": {
            "phoneNumber": "250783456789",
            "provider": "MTN_MOMO_RWA"
        }
    }
}
```

Voyons voir ce que contient cette charge utile.

Nous vous demandons de générer un UUIDv4 `depositId`afin d'identifier ce dépôt de manière unique. Ainsi, vous disposerez toujours d'une référence au dépôt que vous initiez, même si vous ne recevez pas de réponse de notre part en raison d'erreurs réseau. Cela vous permettra de toujours rapprocher tous les paiements entre votre système et pawaPay. Vous devez enregistrer cet identifiant de dépôt dans votre système avant d'initier le dépôt avec pawaPay.

Le `amount`montant `currency`devrait être relativement explicite. Il s'agit de la somme `amount`qui sera perçue auprès du client. Tous les frais seront déduits de ce montant une fois le recouvrement effectué.

Ensuite `provider`, `phoneNumber`précisez qui effectue le paiement. Les portefeuilles de paiement mobile sont identifiés par leur numéro de téléphone, tout comme les comptes bancaires le sont par leur numéro de compte. `provider`Indiquez également l'opérateur de paiement mobile auprès duquel ce portefeuille est enregistré. Par exemple, MTN Ghana ou M-Pesa Kenya.

Vous recevrez une réponse à votre demande.

```
{
```

"depositId": "afb57b93-7849-49aa-babb-4c3ccbfe3d79",
"status": "ACCEPTED",
"nextStep": "FINAL_STATUS",
"created": "2025-05-15T07:38:56Z"
}

Cela `status` indique si ce dépôt était `ACCEPTED` destiné à un traitement ou non. Nous aborderons les modes de défaillance plus loin dans ce guide.

Nous verrons plus tard comment cela `nextStep` peut être utilisé, mais pour l'instant, vous pouvez l'ignorer sans problème.

2

Le client approuvera le paiement

Une fois le dépôt initié, le client recevra un SMS lui demandant de saisir un code PIN pour autoriser le paiement.

Cette étape n'intervient que lors du lancement de paiements réels sur votre compte de production.

Sur votre compte sandbox, cette étape est ignorée !

3

Obtenez le statut final du paiement

Maintenant que le client a approuvé le paiement, vous recevrez un `appel` de notre part à l'URL de rappel que vous avez configurée.

```
{
    "depositId": "afb57b93-7849-49aa-babb-4c3ccbfe3d79",
    "status": "COMPLETED",
    "amount": "100.00",
    "currency": "RWF",
    "country": "RWA",
    "payer": {
      "type": "MMO",
      "accountDetails": {
        "phoneNumber": "250783456789",
        "provider": "MTN_MOMO_RWA"
      }
    },
    "customerMessage": "DEMO",
    "created": "2025-05-15T07:38:56Z",
    "providerTransactionId": "df0e9405-fb17-42c2-a264-440c239f67ed"
  }
```


L'élément principal à retenir ici est le `status`message `COMPLETED`indiquant que le dépôt a été traité avec succès et que les fonds ont été crédités sur votre portefeuille de votre compte pawaPay.

Si vous n'avez pas configuré de rappels, vous pouvez toujours interroger le point de terminaison d'état du dépôt de chèque pour connaître l'état final du paiement.

4

Et voilà !

Félicitations ! Vous avez effectué votre premier dépôt avec pawaPay. Nous avons contacté pawaPay pour initier le dépôt et nous avons pu vérifier son statut final.

Mais une intégration de haute qualité implique bien plus que cela. Dans les sections suivantes, nous aborderons plus en détail :

> Comment faciliter l'expansion vers de nouveaux marchés et fournisseurs ?
> Comment gérer les pannes pour éviter les divergences entre votre système et pawaPay

Comment couvrir les pays avec des fournisseurs qui utilisent différentes méthodes d'autorisation pour les paiements
Et plus encore…

Dans la première partie, nous avons tout codé en dur. Voyons maintenant comment obtenir ces informations auprès du client qui paie.

Nous nous concentrons ici sur le cas où le montant à payer est déjà connu. Plus loin dans ce guide, nous aborderons également le cas où le client peut choisir le montant à payer.

Certains fournisseurs ne prennent pas en charge les décimales dans les montants, de sorte que des montants comme « 100,50 » peuvent ne pas être possibles.

Vous trouverez dans la section « Fournisseurs » des informations sur la prise en charge des décimales dans les montants par les différents prestataires .

1

Afficher les fournisseurs

Le point de terminaison de configuration active contient des informations utiles et importantes, notamment sur les fournisseurs et les marchés configurés sur votre compte pawaPay.

Dans la plupart des cas, vous connaissez déjà le pays d'origine du client. Et comme nous mettons en place les dépôts, nous pouvons récupérer uniquement la configuration relative aux dépôts.

GET
https://api.sandbox.pawapay.io/v2/active-conf?country=RWA&operationType=DEPOSIT

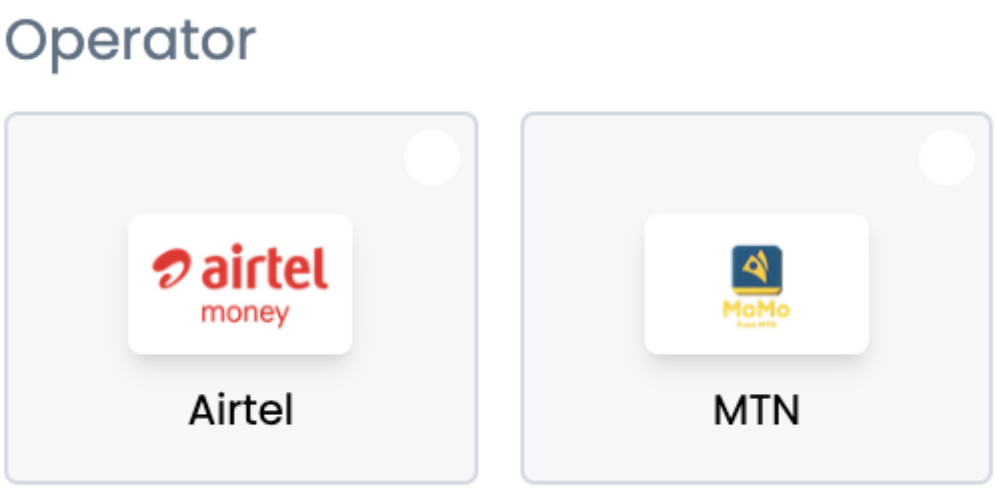

```
{
    "companyName": "Demo",
    "countries": [
        {
```

```
            "country": "RWA",
            "prefix": "250",
            "flag": "https://static-content.pawapay.io/country_flags/rwa.svg",
            "displayName": {
                "en": "Rwanda",
                "fr": "Rwanda"
            },
            "providers": [
                {
                    "provider": "AIRTEL_RWA",
                    "displayName": "Airtel",
                    "logo": "https://static-content.pawapay.io/company_logos/airtel.png",
                    "currencies": [
                        {
                            "currency": "RWF",
                            "displayName": "RF",
                            "operationTypes": ...
                        }
                    ]
                },
                {
                    "provider": "MTN_MOMO_RWA",
                    "displayName": "MTN",
                    "logo": "https://static-content.pawapay.io/company_logos/mtn.png",
                    "currencies": [
                        {
                            "currency": "RWF",
                            "displayName": "RF",
                            "operationTypes": ...
                        }
                    ]
                }
            ]
        }
    ]
    ...
}
```


Comme vous pouvez le constater, `providers`nous avons une liste de tous les fournisseurs configurés sur votre compte pour effectuer des dépôts. Le `displayName`nom du fournisseur y figure. Vous pouvez les présenter comme options à vos clients.

Pour permettre l'ajout automatique de nouveaux prestataires de paiement à vos flux de dépôt, nous avons également intégré leur logo `logo`. Ainsi, le flux de dépôt peut être implémenté de manière dynamique : dès qu'un nouveau prestataire est activé sur votre compte pawaPay, il devient immédiatement disponible pour vos clients, sans intervention supplémentaire.

Cela pourrait ressembler à ceci :



N'indiquez pas de fournisseur par défaut, par exemple en première position dans une liste déroulante. Cette étape est souvent négligée par les clients, ce qui entraîne des échecs de paiement, les fonds étant envoyés au mauvais fournisseur.

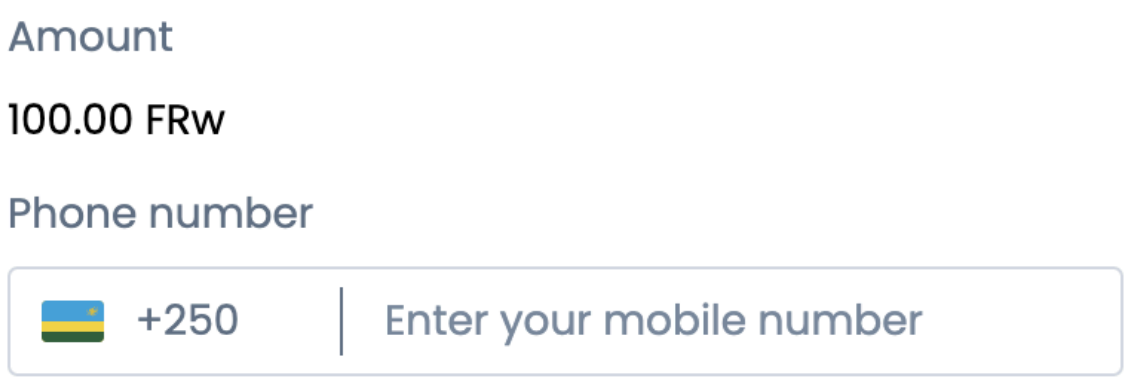
2

Demandez le numéro de téléphone

Dans le point de terminaison de configuration active , l'objet pays contient déjà l' `prefix`indicatif téléphonique du pays. Nous recommandons de l'afficher au client devant le champ de saisie du numéro de téléphone. Cela indique clairement que le numéro saisi ne doit pas inclure l'indicatif du pays.

Nous avons également ajouté des éléments `flag`à l'objet pays pour le rendre un peu plus esthétique.

Cela pourrait ressembler à ceci :



3

Validons maintenant le numéro et identifions le fournisseur.

Pour nous assurer que le numéro saisi par le client est un numéro de téléphone valide, utilisons le point de terminaison du fournisseur de prédiction .

Tout d'abord, concaténez le `prefix`numéro de téléphone saisi par le client.

Vous pouvez maintenant l'envoyer à la validation. Nous nous chargeons des éléments de base comme les espaces, les caractères spéciaux, etc. Nous vérifions également que le nombre de chiffres correspond bien au pays et gérons les zéros non significatifs.

POST https://api.sandbox.pawapay.io/v2/predict-provider

```
{
    "phoneNumber": "25007 834-56789a"
}
```

Si le numéro de téléphone n'est pas valide, une erreur sera renvoyée. Vous pouvez afficher un message d'erreur au client. Sinon, vous obtiendrez la réponse suivante :

```
{
    "country": "RWA",
    "provider": "MTN_MOMO_RWA",
```

```
    "phoneNumber": "250783456789"
}
```

Voici `phoneNumber` le format MSISDN anonymisé du numéro de téléphone que vous pouvez utiliser pour effectuer le dépôt. Nous vous recommandons vivement d'utiliser ce terminal pour la validation des numéros, notamment parce que certains pays n'appliquent pas strictement la norme ITU E.164.

Vous trouverez également dans la réponse le nom `provider` du fournisseur recommandé. Il s'agit du fournisseur prévu pour cette transaction `phoneNumber`. Nous recommandons de présélectionner ce fournisseur pour le client. Dans la plupart des pays, nous constatons une très grande précision de nos prédictions, ce qui simplifie le processus de paiement.

Nous recommandons de permettre au client de modifier la prédiction car sa précision n'est pas de 100 %.

Vous devriez maintenant avoir une page de paiement similaire à celle-ci :

Initier le dépôt

Désormais, toutes les informations proviennent soit du point de terminaison de configuration actif , soit du client, soit de votre système (le montant à payer). Nous pouvons appeler le point de terminaison d'initiation de dépôt avec ces informations.

POST https://api.sandbox.pawapay.io/v2/deposits

```
{
```

"depositId": "afb57b93-7849-49aa-babb-4c3ccbfe3d79",

"amount": "100",

"currency": "RWF",

"payer": {

"type": "MMO",

"accountDetails": {

"phoneNumber": "250783456789",

"provider": "MTN_MOMO_RWA"

}

}

}

La réponse et le rappel avec le statut final sont identiques à ceux de la première partie du guide.

Veuillez noter que certains fournisseurs ne prennent pas en charge les montants décimaux. Vous trouverez la liste des fournisseurs qui les prennent en charge dans la section « Fournisseurs » .

Refait !

Nous avons désormais mis en place un flux de dépôt réaliste, collectant des informations provenant de différentes sources et évitant toute donnée codée en dur. Ainsi, l'ajout de nouveaux fournisseurs n'entraînera aucune modification supplémentaire pour vous.

Voyons maintenant ce qu'il convient de faire en attendant que le client autorise le paiement.

Une fois le dépôt initié, le client devra l'autoriser. Plusieurs méthodes d'autorisation de paiement existent, mais la plus courante consiste à afficher une demande de code PIN sur le téléphone du client. Nous nous concentrerons ici sur ce type d'autorisation. Les autres méthodes seront abordées plus loin dans ce guide.

1

Trouver le type d'autorisation de paiement

Nous avons déjà utilisé le point de terminaison de configuration active pour identifier les fournisseurs configurés sur votre compte pawaPay. Le type d'autorisation utilisé par le fournisseur est également disponible via ce point de terminaison.

GET
https://api.sandbox.pawapay.io/v2/active-conf?country=SLE&operationType=DEPOSIT


```
{
    "companyName": "Demo",
    "countries": [
        {
            ...
            "providers": [
                {
                    "provider": "ORANGE_SLE",
                    "displayName": "Airtel",
                    "nameDisplayedToCustomer": "SL LIMITED",
                    "logo":
"https://static-content.pawapay.io/company_logos/orange.png",
                    "currencies": [
                        {
                            "currency": "SLE",
```

```
                "displayName": "Le",
                "operationTypes": {
                    "DEPOSIT": {
                        ...
                        "authType": "PROVIDER_AUTH",
                        "pinPrompt": "MANUAL",
                        "pinPromptRevivable": true
                        ...
                        }
                    ]
            }
        ]
    }
  ]
  ...
}
```

Nous allons nous concentrer sur trois propriétés ici.

Ce `authType`symbole indique le type d'autorisation utilisé par ce fournisseur. Nous nous concentrons `PROVIDER_AUTH`actuellement sur celui-ci. Les autres `authTypes`seront abordés ultérieurement dans ce guide.

Cette `pinPrompt`propriété s'applique uniquement aux fournisseurs disposant `authType`de `PROVIDER_AUTH`. Elle indique si l'invite de code PIN faisant partie de l'autorisation apparaîtra automatiquement sur le téléphone du client (`AUTOMATIC`) ou s'il devra effectuer une action à cet effet (`MANUAL`).

Cela `pinPromptRevivable`indique si le client peut faire réapparaître l'invite de code PIN en cas d'échec lors de la première tentative.

Maintenant que nous connaissons les détails du flux d'autorisation du fournisseur, implémentons l'écran à afficher au client après avoir initié le dépôt.

2

Expérience client pour l'invite de code PIN AUTOMATIQUE

En `pinPrompt`contrepartie, `AUTOMATIC`nous pouvons afficher au client un écran indiquant qu'il doit autoriser le paiement en saisissant son code PIN de paiement mobile sur son téléphone.

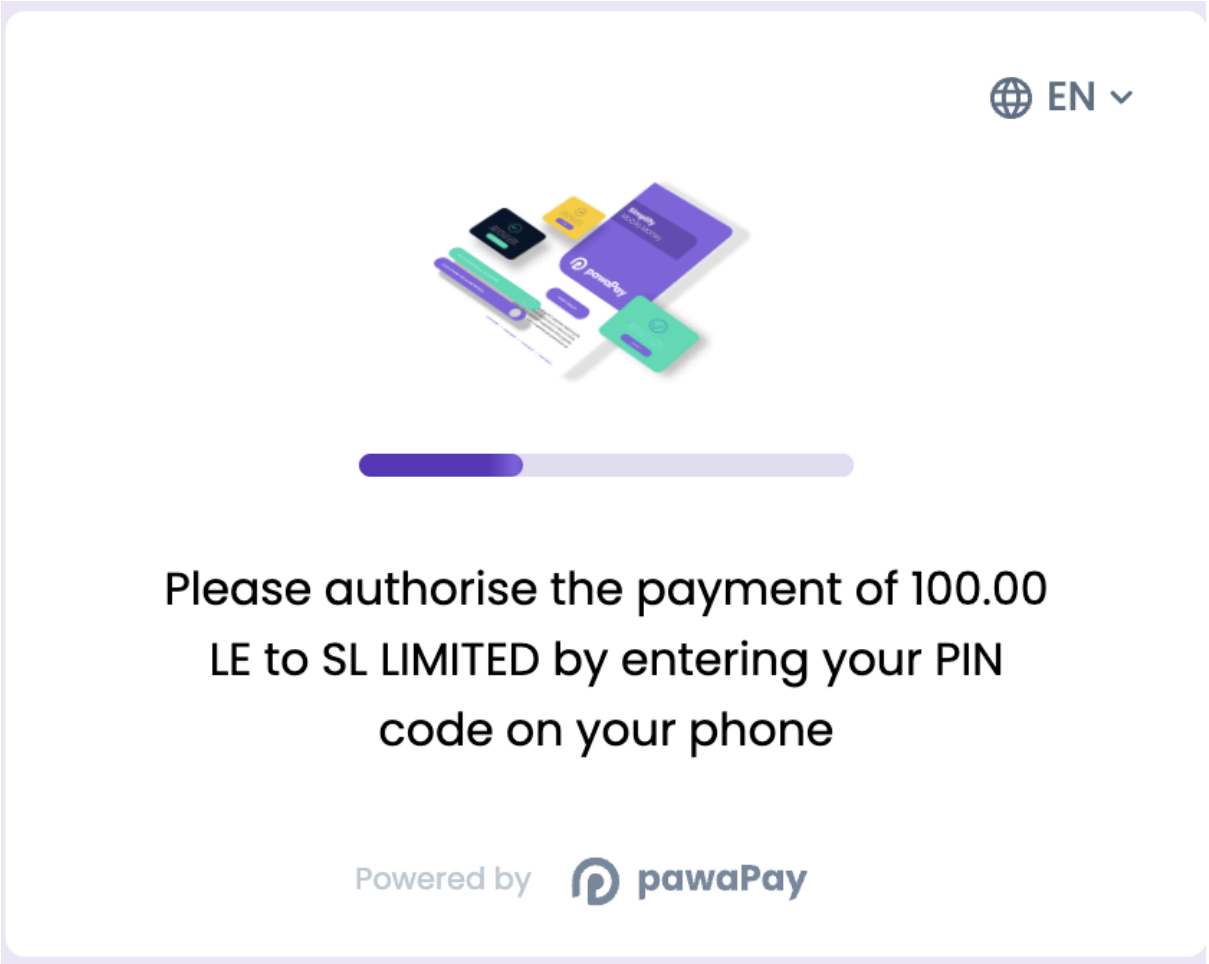
Lors de la saisie du code PIN, une référence au prestataire de paiement (pawaPay ou l'une de ses filiales locales) est généralement indiquée. Vous devez la communiquer au client dans le cadre du message afin de confirmer la validité de la demande d'autorisation de paiement. Cette information figure dans la réponse `nameDisplayedToCustomer`.

GET https://api.sandbox.pawapay.io/v2/active-conf?country=SLE&operationType=DEPOSIT


```
{
    "companyName": "Demo",
    "countries": [
        {
            ...
            "providers": [
                {
                    "provider": "ORANGE_SLE",
                    "displayName": "Airtel",
                    "nameDisplayedToCustomer": "SL LIMITED",
                    ...
                }
            ]
        }
    ]
```

```
    ...

  }
```

Voici un exemple de ce que vous pourriez montrer au client en attendant le rappel concernant le statut final de ce paiement.



3

Et si l'invite de code PIN était réactivable ?

Si `pinPromptRevivable`cela est vrai pour le fournisseur, cela signifie que le client peut relancer l'invite de code PIN en cas de problème lors de la première tentative. Par exemple, cela peut se produire s'il ne retrouve pas son téléphone avant l'expiration du délai d'affichage de l'invite de code PIN, s'il reçoit un appel simultanément ou en cas de problème de réseau empêchant l'affichage de l'invite de code PIN.

Dans ce cas, nous recommandons d'attendre 10 à 15 secondes après le lancement et d'afficher au client les instructions pour faire réapparaître l'invite de code PIN et réessayer. Ces instructions se trouvent dans la réponse du point de terminaison de configuration active sous le nom « pinPromptInstructions ».


```
...
  "pinPromptInstructions": {
    "channels": [
      {
        "type": "USSD",
        "displayName": {
          "en": "Did not get a PIN prompt?",
          "fr": "N’avez-vous pas reçu la demande de code PIN?"
        },
        "instructions": {
```

```
"en": [
    {
        "text": "Dial *115#",
        "template": "Dial {{shortCode}}",
        "variables": {
            "shortCode": "*115#"
        }
    },
    {
        "text": "Enter PIN",
        "template": "Enter PIN",
        "variables": {
            "shortCode": "*115#"
        }
    }
],
"fr": [
    {
```

"text": "Composez *115#",
"template": "Composez {{shortCode}}",
"variables": {
"shortCode": "*115#"
}
},
{
"text": "Entrez le code PIN",
"template": "Entrez le code PIN",
"variables": {
"shortCode": "*115#"
}
}
]
}
}
]
},

...

L' `channel`indication permettant de faire réapparaître l'invite de code PIN précise où le client peut la faire réapparaître. Cela `USSD`signifie généralement qu'il doit composer un code USSD sur son téléphone.

Le `displayName`document contient un message expliquant l'objectif des instructions. Vous pouvez l'utiliser ou rédiger votre propre message.

L' `quickLink`attribut `href` (si disponible) contient le lien permettant de pré-composer le code USSD. Utilisez-le comme attribut ` href` `href`d'une `<a />`balise `<a>` ou d'un bouton.

Ainsi, lorsque le client utilise le même téléphone pour visiter votre site et effectuer le paiement, il peut appuyer sur ce lien pour pré-composer le code USSD.

Ces instructions `instructions`incluent les étapes exactes à suivre pour faire réapparaître le code PIN. Vous pouvez les parcourir pour afficher les instructions de réapparition de l'invite de code PIN. Vous pouvez utiliser `text`directement le texte de chaque instruction ou, si vous souhaitez mettre en évidence des propriétés clés, utiliser les balises <code> `template`et <code> `variables`. Il suffit d'encadrer le texte `variables`d'une instruction par l'élément à mettre en évidence et de le remplacer dans le modèle.

4

Expérience client pour MANUEL `pinPrompt`

En cas de `pinPrompt`paiement `MANUAL`, le client doit effectuer une action pour accéder à l'invite de saisie du code PIN. Bien que la plupart des clients sachent quoi faire et reçoivent de toute façon un SMS contenant les instructions, nous avons constaté

une amélioration du taux de réussite grâce à des instructions claires intégrées au processus de paiement.

Vous pouvez obtenir les instructions pour cela exactement de la même manière que pour la réactivation de l'invite de code PIN.


```
  "pinPromptInstructions": {
      "channels": [
        {
            "type": "USSD",
            "displayName": {
                "en": "Follow the instructions to authorise the payment",
                "fr": "Veuillez suivre les instructions afin d’autoriser le paiement."
            },
            "instructions": {
                "en": [
                    {
                        "text": "Dial *555*6#",
                        "template": "Dial {{shortCode}}",
                        "variables": {
                            "shortCode": "*555*6#"
```

```
        }
    },
    {
        "text": "Enter PIN",
        "template": "Enter PIN",
        "variables": {
            "shortCode": "*555*6#"
        }
    }
],
"fr": [
    {
        "text": "Composez *555*6#",
        "template": "Composez {{shortCode}}",
        "variables": {
            "shortCode": "*555*6#"
        }
    },
```

```
                {
                    "text": "Entrez le code PIN",
                    "template": "Entrez le code PIN",
                    "variables": {
                        "shortCode": "*555*6#"
                    }
                }
            ]
        }
    }
  ]
},
```

Et voilà !

Nous nous sommes désormais assurés que le client sache toujours ce qu'il doit faire pour autoriser un paiement.

Voyons maintenant comment gérer les échecs.

L'initiation d'un paiement peut échouer pour diverses raisons. Voyons comment gérer au mieux ces situations.

1

Gestion des interruptions de service du fournisseur

Les fournisseurs peuvent connaître des interruptions de service. Nous assurons une surveillance continue de leurs performances et de leur disponibilité, 24 h/24 et 7 j/7. Une page d'état est mise à la disposition de vos équipes opérationnelles et de support . Elles peuvent s'y abonner pour recevoir des mises à jour par e-mail ou Slack concernant les fournisseurs qui les intéressent.

Pour éviter les échecs de paiement, nous avons également rendu ces informations accessibles via l'API, à partir des points de terminaison de disponibilité du fournisseur et de configuration active . Ainsi, vous pouvez informer le client en amont, avant même qu'il ne tente de payer.

GET
https://api.sandbox.pawapay.io/v2/active-conf?country=BEN&operationType=DEPOSIT


```
{
    "companyName": "Demo",
    "countries": [
```

```
{
    "country": "BEN",
    "displayName": {
        "fr": "Bénin",
        "en": "Benin"
    },
    "prefix": "229",
    "providers": [
        {
            "provider": "MOOV_BEN",
            "nameDisplayedToCustomer": "PAWAPAY",
            "currencies": [
                {
                    "currency": "XOF",
                    "displayName": "CFA",
                    "operationTypes": {
                        "DEPOSIT": {
                            ...
```

```
                "status": "OPERATIONAL",
                ...
            }
        }
    }
  ]
},
{
  "provider": "MTN_MOMO_BEN",
  "nameDisplayedToCustomer": "PAWAPAY",
  "currencies": [
    {
        "currency": "XOF",
        "displayName": "CFA",
        "operationTypes": {
            "DEPOSIT": {
                ...
                "status": "CLOSED",
```

```
                    ...
                  }
                }
              }
            ]
          }
        ]
      }
    ]
    ...
  }
```

Selon la disponibilité `status`du fournisseur `OPERATIONAL`, `CLOSED`vous pouvez informer le client dès le départ que ce fournisseur n'est pas disponible actuellement et qu'il pourra réessayer plus tard.

Gestion des décimales dans les montants et les limites de transaction

Chaque fois que vous initiez un dépôt, vous devez confirmer que la valeur `status`dans la réponse est `ACCEPTED`.

Si le statut est `REJECTED`celui `failureReason`indiqué, il contiendra à la fois l'information `failureCode`et `failureMessage`ce qui s'est passé. La plupart de ces défaillances sont évitables si elles sont gérées en amont.

L' `failureMessage`API pawaPay est conçue pour vous et vos équipes de support et d'exploitation. Vous êtes libre de choisir le message à afficher au client.

```
//Incorrect amount of decimals in amount
{
    "depositId": "f4401bd2-1568-4140-bf2d-eb77d2b2b639",
    "status": "REJECTED",
    "failureReason": {
        "failureCode": "INVALID_AMOUNT",
        "failureMessage": "The provider MOOV_BEN only supports up to '2' decimal places in amount."
    }
}

//Amount larger than transaction limits allow
```

{

"depositId": "f4401bd2-1568-4140-bf2d-eb77d2b2b639",

"status": "REJECTED",

"failureReason": {

"failureCode": "AMOUNT_OUT_OF_BOUNDS",

"failureMessage": "The amount needs to be more than '100' and less than '2000000' for provider 'MOOV_BEN'."

}

}

//Invalid currency

{

"depositId": "f4401bd2-1568-4140-bf2d-eb77d2b2b639",

"status": "REJECTED",

"failureReason": {

"failureCode": "INVALID_CURRENCY",

"failureMessage": "The currency 'USD' is not supported with provider 'MOOV_BEN'. Please consult API docs for supported currencies."

}

```
}
```

Dans l'exemple ci-dessus, si le fournisseur ne prend pas en charge les décimales dans les montants, vous devez anticiper ce problème en arrondissant les montants. Selon votre cas d'utilisation, vous pouvez le faire de manière préventive en ajustant les prix locaux ou dynamiquement avant le paiement.

Deuxièmement, il faut savoir que les portefeuilles mobiles des clients ont des limites quant au montant des paiements. Les clients peuvent augmenter ces limites en contactant leur fournisseur et en effectuant une vérification d'identité approfondie (KYC). Par défaut, nous avons configuré les limites de transaction de votre compte pawaPay en fonction des limites les plus courantes des portefeuilles mobiles.

Concernant les devises, seul le RDC est disponible via pawaPay et prend en charge plusieurs devises . Celles-ci sont exposées dans le point de terminaison de configuration active sous forme de tableau `currencies`.

Vous pouvez accéder à la fois au nombre de décimales prises en charge et aux limites de transaction à partir du point de terminaison de configuration actif .

GET
https://api.sandbox.pawapay.io/v2/active-conf?country=BEN&operationType=DEPOSIT

```
{
    "companyName": "Demo",
```

"countries": [
  {
    "country": "BEN",
    "displayName": {
      "fr": "Bénin",
      "en": "Benin"
    },
    "prefix": "229",
    "providers": [
      {
        "provider": "MOOV_BEN",
        "nameDisplayedToCustomer": "PAWAPAY",
        "currencies": [
          {
            "currency": "XOF",
            "displayName": "CFA",
            "operationTypes": {
              "DEPOSIT": {

```
                ...
                "decimalsInAmount": "NONE",
                "minAmount": "100",
                "maxAmount": "2000000",
                ...
            }
        }
    }
  ]
},
{
  "provider": "MTN_MOMO_BEN",
  "nameDisplayedToCustomer": "PAWAPAY",
  "currencies": [
    {
        "currency": "XOF",
        "displayName": "CFA",
        "operationTypes": {
```

```
                "DEPOSIT": {
                    ...
                    "decimalsInAmount": "NONE",
                    "minAmount": "100",
                    "maxAmount": "2000000",
                    ...
                }
            }
        }
    ]
  }
]
...
}
```

Vous pouvez les utiliser pour vous assurer que les données d'entrée nécessaires au dépôt sont nettoyées avant le lancement et pour éviter les rejets.

3

Assurez-vous que le format du numéro de téléphone est correct.

Le point de terminaison de dépôt attend le `phoneNumber` format MSISDN.

Vous pouvez utiliser le point de terminaison du fournisseur de prédiction pour nettoyer les données saisies par le client. Assurez-vous que ces données commencent par l'indicatif du pays.

POST https://api.sandbox.pawapay.io/v2/predict-provider

```
{
    "phoneNumber": "25007 834-56789a"
}
```

Si le numéro de téléphone n'est pas valide, une erreur sera renvoyée. Vous pouvez alors afficher un message d'erreur au client.

```
{
    "country": "RWA",
    "provider": "MTN_MOMO_RWA",
    "phoneNumber": "250783456789"
}
```

La `phoneNumber`réponse devra être au format correct pour initier le dépôt .

4

Une seule initiation par identifiant de dépôt

Tous les paiements effectués avec pawaPay sont idempotents grâce à leur identifiant. Vous devez générer un nouvel UUIDv4 pour chaque transaction `depositId`. Toute tentative d'utilisation `depositId`répétée du même UUID entraînera le rejet du paiement.

```
{
    "depositId": "f4401bd2-1568-4140-bf2d-eb77d2b2b639",
    "status": "DUPLICATE_IGNORED"
  }
```

Disponibilité des prestataires de manutention

Nous avons déjà évoqué la manière d'indiquer au client les modes de paiement actuellement indisponibles en raison d'une interruption de service chez le fournisseur. Un certain délai s'écoule généralement entre le moment où vous affichez ces informations au client et le moment où il initie le paiement. Il est donc essentiel de veiller à ce que les informations affichées restent exactes, même si le fournisseur est indisponible pendant ce laps de temps.

Si un paiement est initié pendant une interruption de service du fournisseur, il sera rejeté.

{

"depositId": "f4401bd2-1568-4140-bf2d-eb77d2b2b639",

"status": "REJECTED",

"failureReason": {

"failureCode": "PROVIDER_TEMPORARILY_UNAVAILABLE",

"failureMessage": "The provider 'MTN_MOMO_BEN' is currently not able to process payments. Please consult our status page for downtime information on all providers. Programmatic access is also available, please consult our API docs."

}

}

De nombreux clients possèdent des portefeuilles de paiement mobile auprès de différents opérateurs. Il est donc raisonnable de leur proposer de payer avec un autre opérateur ou de réessayer plus tard.

6

Gestion de l'erreur HTTP 500 avec le code d'erreur UNKNOWN_ERROR

Le `UNKNOWN_ERROR`code d'erreur indique qu'un problème inattendu est survenu lors du traitement du paiement. Aucune erreur n'est signalée `status`dans la réponse dans ce cas.

Il est imprudent de supposer que l'initiation de ce dépôt a échoué. Veuillez vérifier le statut du dépôt à l'aide du point de terminaison de suivi des dépôts . Le dépôt ne `NOT_FOUND`doit être considéré comme validé que s'il est validé `FAILED`.

Nous verrons plus loin dans ce guide comment garantir la cohérence des statuts de paiement entre votre système et pawaPay.

7

Et voilà !

Nous avons désormais géré différents types de défaillances pouvant survenir lors de l'initialisation.

Passons maintenant aux échecs de paiement qui peuvent survenir lors du traitement.

1

Gestion des erreurs lors du traitement

L'API pawaPay étant asynchrone, vous recevrez une confirmation de dépôt avec le statut final. En cas `status`d'échec, `FAILED`vous trouverez des informations supplémentaires à ce sujet dans la documentation `failureReason`. Celle-ci inclut le code d' erreur `failureCode`et le message d' `failureMessage`erreur indiquant la cause du problème.

L' `failureMessage`API pawaPay est conçue pour vous et vos équipes de support et d'exploitation. Vous êtes libre de choisir le message à afficher au client.

Identifiez tous les codes d'erreur et implémentez la gestion de votre choix.

Les échecs de traitement spécifiques à chaque opération sont également documentés dans la référence de l'API :

- Rappel de dépôt
- Rappel de paiement
- Rappel de remboursement

Nous vous recommandons de faciliter les nouvelles tentatives de paiement en redirigeant le client vers la page de saisie des informations de paiement et en y affichant le motif de l'échec. Il pourra ainsi réessayer rapidement.

Nous avons standardisé les nombreux codes de défaillance et scénarios différents avec tous les différents fournisseurs.

La spécificité des codes d'échec varie selon le fournisseur. Ce `UNSPECIFIED_FAILURE`code indique que le fournisseur a signalé un échec de paiement, sans toutefois préciser la raison de cet échec.

En cas de défaillance générale, le `UNKNOWN_ERROR`code de défaillance sera renvoyé.

2

Et voilà !

Nous avons également pris en compte les problèmes pouvant survenir lors du traitement des paiements. Ainsi, le client est informé de ce qui s'est passé et peut prendre les mesures nécessaires pour réessayer.

Examinons maintenant quelques marchés où le flux d'autorisation des paiements est légèrement différent de d'habitude.

Lorsqu'on utilise des API financières, il est important de prendre certaines précautions afin d'éviter de considérer un paiement comme ayant échoué alors qu'il a réussi, et inversement. Il est essentiel de synchroniser les systèmes concernant le statut des paiements.

Examinons quelques considérations et un pseudocode pour garantir la cohérence.

1

Gestion du statut défensif

Tous les statuts doivent être vérifiés de manière préventive, sans faire de suppositions.

```
if( status == "COMPLETED" ) {

        myInvoice.setPaymentStatus(COMPLETED);

    } else if ( status == "FAILED" ) {

        myInvoice.setPaymentStatus(FAILED);

    } else if  ( status == "PROCESSING") {

        handleRedirectionAuth();

    } else {
```

```
        //It is unclear what might have failed. Escalate for further investigation.

        myInvoice.setPaymentStatus(NEEDS_ATTENTION);

    }
```

Gestion des erreurs réseau et des pannes système

La principale raison pour laquelle nous vous demandons de fournir un justificatif `depositId`pour chaque paiement est de vous permettre de toujours nous interroger sur le statut d'un paiement, même si vous ne recevez jamais de réponse de notre part.

Vous devez toujours enregistrer ces informations `depositId`dans votre système avant d'effectuer un dépôt .

```
  var depositId = new UUIDv4();



    //Let's store the depositId we will use to ensure we always have it available even if something dramatic happens

    myInvoice.setExternalPaymentId(depositId).save();

    myInvoice.setPaymentStatus(PENDING);



    try {

        var initiationResponse = pawaPay.initiateDeposit(depositId, ...)
```

```
} catch (InterruptedException e) {
    var checkResult = pawaPay.checkDepositStatus(depositId);


    if ( result.status == "FOUND" ) {
        //The payment reached pawaPay. Check the status of it from the response.
    } else if ( result.status == "NOT_FOUND" ) {
        //The payment did not reach pawaPay. Safe to mark it as failed.
        myInvoice.setPaymentStatus(FAILED);
    } else {
        //Unable to determine the status. Leave the payment as pending.
        //We will create a status recheck cycle later for such cases.


        //In case of a system crash, we should also leave the payment in pending status to be handled in the status recheck cycle.
    }
}
```

Il est important de noter que nous ne signalons un paiement comme ÉCHOUÉ que lorsqu'il existe une indication claire de son échec. Nous utilisons le point de

terminaison de statut de dépôt de chèque en cas de doute quant à savoir si le paiement a été effectué `ACCEPTED`par pawaPay.

3

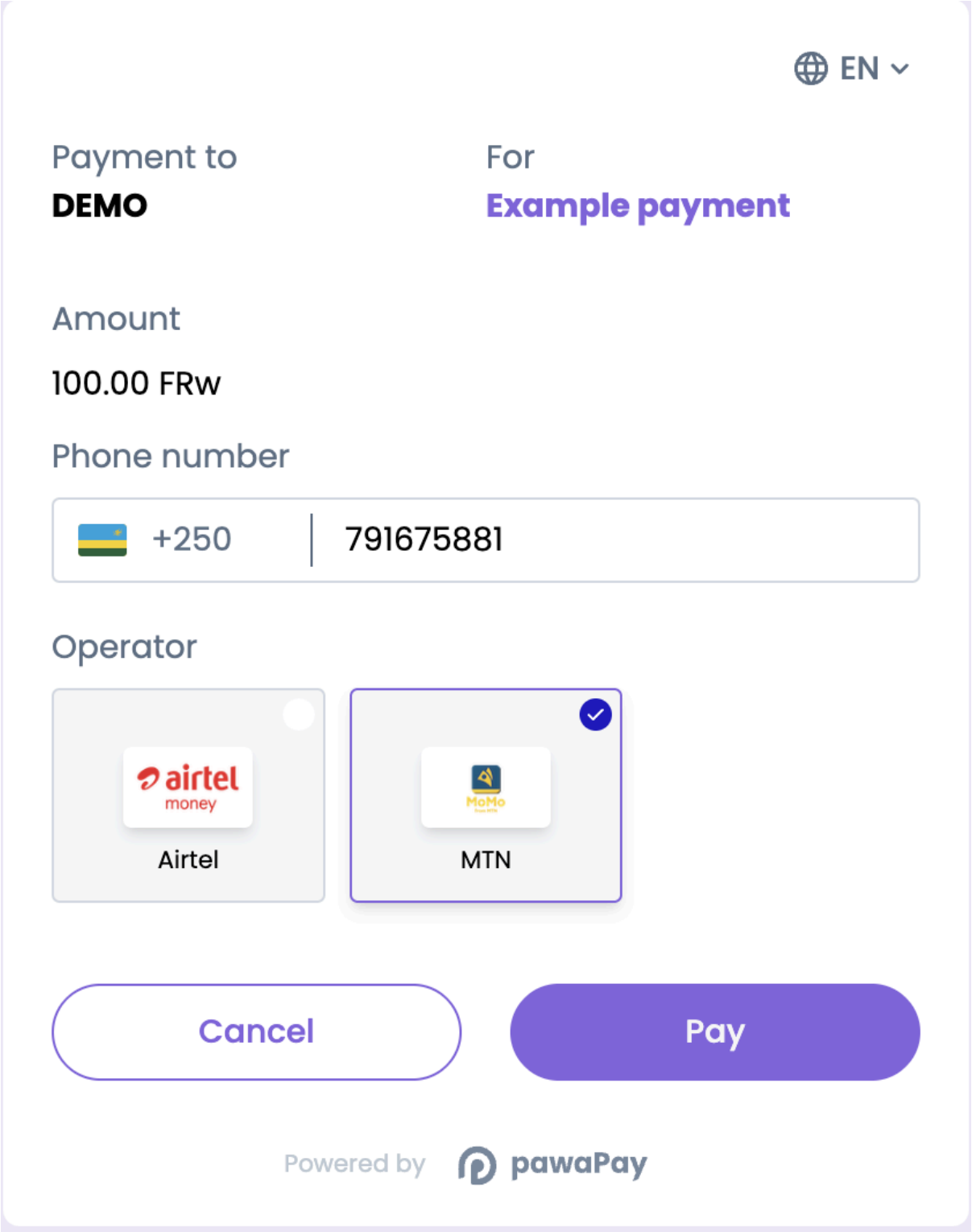
Mise en œuvre d'un cycle de rapprochement automatisé

La mise en œuvre des recommandations ci-dessus permet d'éviter la quasi-totalité des divergences de statut de paiement entre votre système et pawaPay. Lors de l'utilisation de notifications pour recevoir le statut final des paiements, des problèmes tels que des interruptions de service, des pannes réseau ou des erreurs de configuration peuvent empêcher la réception de ces notifications par votre système. Afin d'éviter toute attente pour vos clients, nous vous recommandons vivement de mettre en place un cycle de vérification du statut.

Cela pourrait ressembler à ceci.

```
//Run the job every few minutes.



    var pendingInvoices = invoices.getAllPendingForLongerThan15Minutes();



    for ( invoice in pendingInvoices ) {

        var checkResult =
pawaPay.checkDepositStatus(invoice.getExternalPaymentId);



        if ( checkResult.status == "FOUND" ) {
```

```
        //Determine if the payment is in a final status and handle accordingly

        handleInvoiceStatus(checkResult.data);

    } else if (checkResult.status == "NOT_FOUND" ) {

        //The payment has never reached pawaPay. Can be failed safely.

        invoice.setPaymentStatus(FAILED);

    } else {

        //Something must have gone wrong. Leave for next cycle.

    }

}
```



4

Initier le dépôt

Désormais, toutes les informations proviennent soit du point de terminaison de configuration actif , soit du client, soit de votre système (le montant à payer). Nous pouvons appeler le point de terminaison d'initiation de dépôt avec ces informations.